Planner Comercial

Desenvolvimento, funcionamento e próximos passos

Atualização de 16 de outubro de 2026

O projeto já permite demonstrar um fluxo integrado: criar uma conta, registrar uma venda, acompanhar a separação dos valores, consultar o histórico, registrar despesas e conferir o caixa. O painel web conversa com uma API Java, que grava os dados no PostgreSQL.

O banco é real; o processamento financeiro é demonstrativo. Os registros permanecem após reiniciar o sistema. Pagamentos, destinos e tributos são simulados: o aplicativo não transfere dinheiro nem comprova recolhimento fiscal.

O que foi acrescentado

Primeiro acesso: cadastro de uma nova loja e gerente, login e edição do nome do estabelecimento.

Venda e recuperação: regra histórica preservada e recuperação de tentativas sem duplicar uma venda ou um pagamento.

Planejamento: simulação com percentual hipotético, projeção manual de entradas e despesas e aviso de déficit previsto.

Consulta: relatórios de vendas, despesas e tributos, com período e CSV; caixa semanal com saldos conciliados e semanas sem movimento.

Explicações: composição de cada split, leitura do caixa e comparação entre períodos, sem inventar dados quando a base está vazia.

Apresentação local: carga opcional de vendas e despesas sintéticas em duas lojas, com estados diferentes e repetição sem duplicar registros.

Como interpretar a entrega

O núcleo funciona localmente e recebeu testes. Isso ainda não significa que todos os 516 critérios foram aceitos ou que o sistema esteja pronto para uso financeiro real. Os limites e as verificações restantes aparecem na última página.

Como o sistema foi construído

Três aplicações, responsabilidades separadas

Painel web - React e TypeScript. É a interface aberta no navegador. Vite prepara e executa o projeto; CSS organiza o visual e a adaptação às larguras de tela. As telas ficam em **frontend/src/telas**, e os componentes reutilizáveis, em **frontend/src/componentes**.

Servidor - Java 21 e Spring Boot. A API valida acesso, loja, entradas e regras, calcula a composição e registra operações. O código fica em **backend/src/main/java**. O navegador não acessa o banco diretamente.

Aplicativo Android - Kotlin. O POS possui demonstração local e cliente preparado para servidor HTTPS. O código fica em **android-pos/app/src/main**. Compilar o APK não substitui a homologação em aparelho.

Exemplos para localizar o código

TelaRegistroVenda.tsx: formulário, revisão e confirmação da venda no site.

contratoBackendPlanner.ts: comunicação e adaptação dos dados recebidos do servidor.

ServicoRegistroVenda.java: registro e recuperação da intenção de venda.

ServicoExplicacaoFinanceira.java: explicação do caixa com limites de espera e resposta alternativa em caso de falha.

TelaPosIntegradoActivity.kt: interface Android conectada ao servidor.

Banco e proteção dos registros

O **PostgreSQL** guarda usuários, lojas, vendas, parcelas, despesas, sessões e tentativas. O **Flyway** aplica as mudanças de estrutura por migrations versionadas. Senhas são protegidas com hash; o servidor verifica o perfil e a loja autorizada em cada ação.

Idempotência significa reconhecer uma tentativa repetida. Se a resposta se perder e o usuário tentar novamente, o sistema recupera a mesma intenção em vez de gerar outra venda.

Resultados e o que falta

O que foi verificado nesta revisão

Web: 100 testes aprovados e compilação concluída.

Backend: 61 testes aprovados, incluindo banco, recuperação e carga de dados sintéticos sem duplicação.

Android: testes locais, compilação do APK e análise estática concluídos.

Navegador com API: 13 cenários anteriores aprovados e mais uma verificação do caixa semanal, incluindo navegação, largura de tela e acessibilidade automática.

Layout: nove telas verificadas em cinco larguras, de 320 a 1440 pixels. A análise automática apontou contraste insuficiente nas mensagens de estado; a correção passou no reteste.

Consultas simultâneas: 120 solicitações com oito em paralelo, sem falhas neste computador. É uma medição curta, não um teste de capacidade de produção.

Pendências que permanecem explícitas

Requisitos: os 516 critérios têm revisão individual com referências e limites. Restam aceites e verificações apontados na matriz, incluindo a comparação visual completa com o Figma.

Infraestrutura: Docker, reinício e restauração foram aprovados no GitHub, com 15 tabelas conferidas. Hospedagem, HTTPS e banco compartilhado ainda precisam ser configurados.

Android e acessibilidade: homologar o aplicativo em aparelho com HTTPS, TalkBack e uso real; completar verificações manuais de foco, zoom e estados de interface.

Operação e segurança: logs e métricas locais preparados; dependências corrigidas. Dois alertas do Spring permanecem sobre recursos não usados. Definir responsáveis, retenção e ambiente compartilhado.

IA externa é opcional. A versão atual usa explicações demonstrativas baseadas nos dados autorizados do servidor. A equipe não precisa contratar um modelo para apresentar esse funcionamento. Crédito e cancelamento continuam previstos como evolução futura.

Design, prototipação e pesquisa: Thayná Batista da Silva. Requisitos: Thayná Batista da Silva e Kleysson Teixeira, conforme materiais fornecidos. Código e evidências: github.com/lucasandare490/Avanade.